



山东大学

信息科学与工程学院

2025—2026 学年第一学期

实 验 报 告

课程名称: Java 编程技术

实验名称: 第一次实验

专 业 班 级 通信三班

学 生 学 号

学 生 姓 名

实 验 时 间 2025 年 9 月 18 日

实 验 报 告

【实验目的】

1. 掌握安装 SDK 软件包、Eclipse 软件、EditPlus 编辑软件的方法。
2. 掌握设置程序运行环境的方法。
3. 掌握编写与运行程序的方法。
4. 理解面向对象的编程思想。

【实验要求】

编写一个简单的控制台应用程序，该程序在命令行窗口输出两行文字：“Hello World!”和“We are students.”。

- 1) 使用 Eclipse 软件，编译并显示实现。
- 2) 用命令行方式，首先将自己写的文件编译成 class 文件，如下图所示：

【实验具体内容】

(1) 实验源代码（粘贴源代码）：

```
public class HelloWorld {
    public static void main(String[] args ) {
        system.out.println("Hello World!!!");
        system.out.println("We are students!!!");
    }
}
```

(2) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

- 1) 利用 Notepad++编写代码，并将其后缀改为/. java 格式。

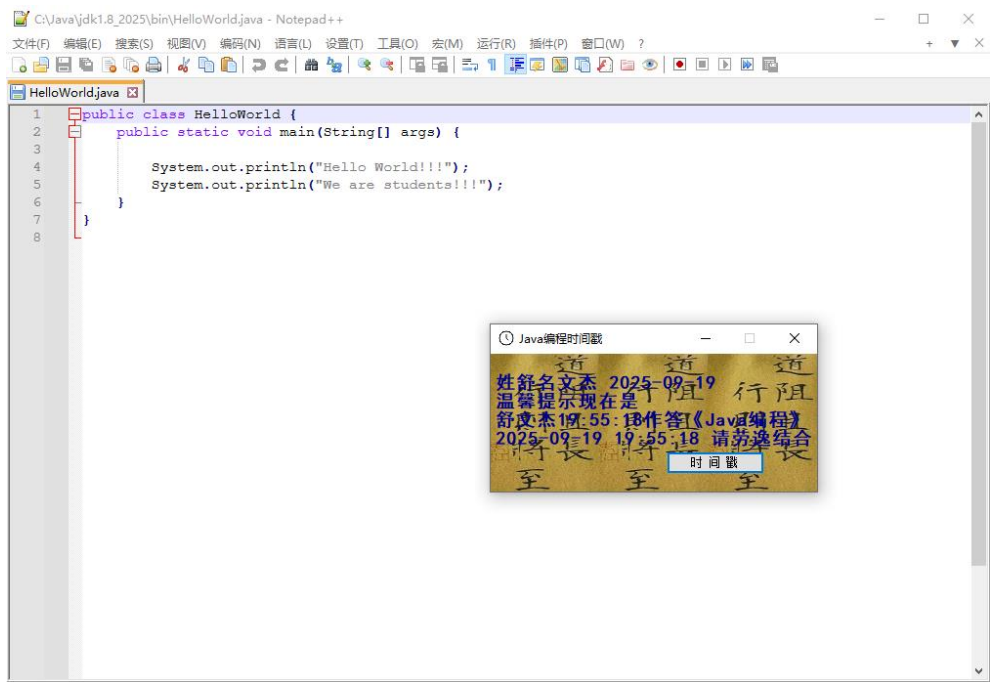


图 1 编写 java 程序

2) 将其保存在 C:\Java\jdk1.8_2025 路径下，使用命令提示窗编译运行程序

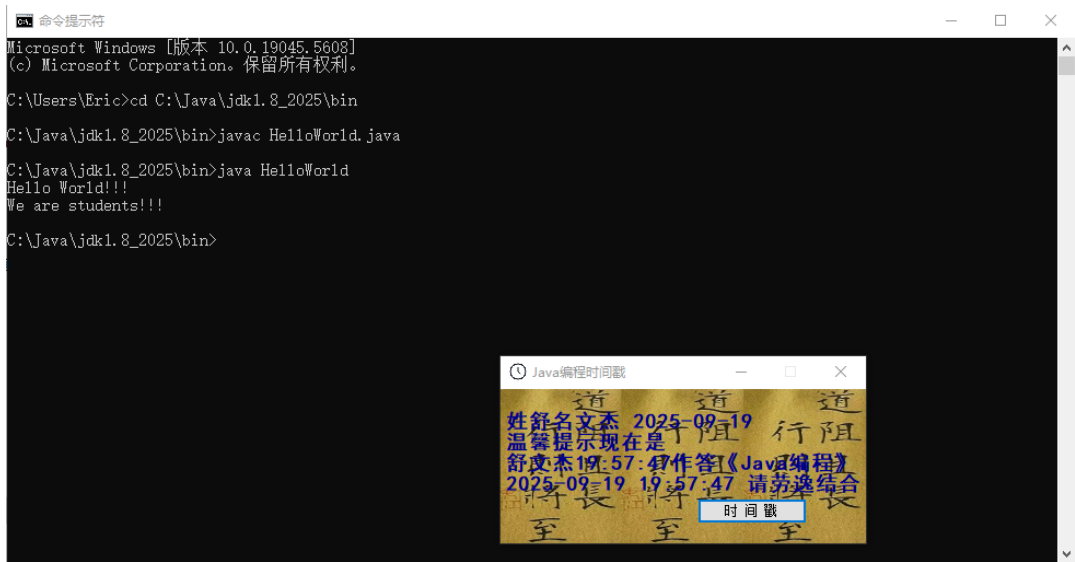


图 2 运行情况截图

 HelloWorld.class	2025/9/19 19:57	CLASS 文件	1 KB
 HelloWorld.java	2025/9/18 13:33	Java 源文件	1 KB

图 3 编译效果截图

可以发现，我们成功编译并运行了代码，看来 Java 环境我们配置得没有问题，于

是我们进行 eclipse 软件的运用。

3) 复制如上代码并直接在 eclipse 软件直接运行

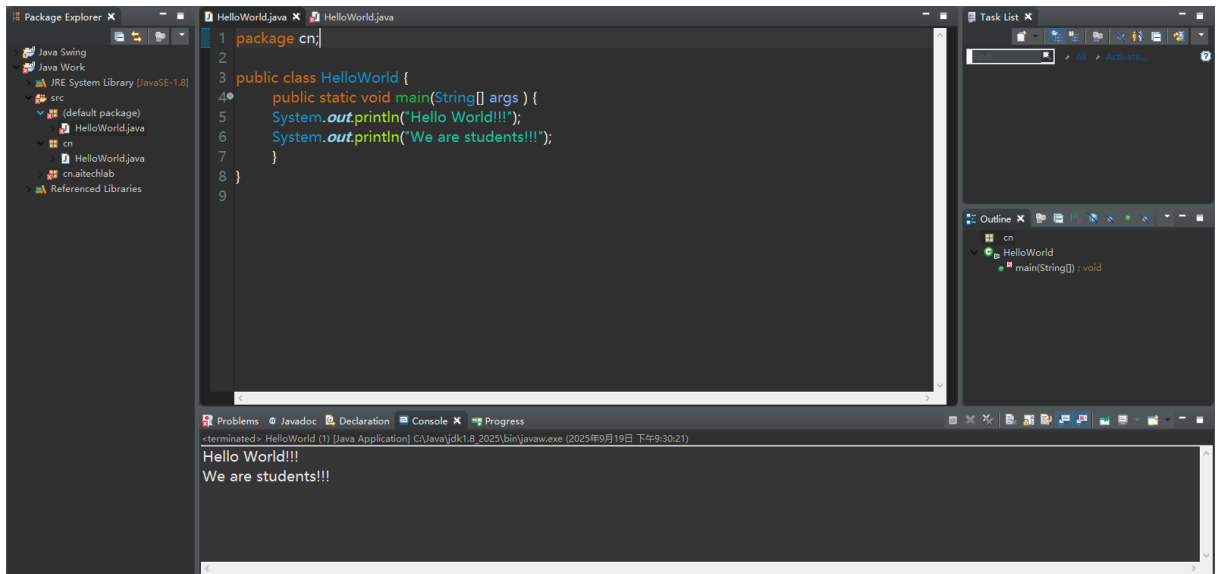


图 4 运行成功代码截图

我们的代码成功运行，但是有一个地方令我感到好奇，为什么在我创建文件时会自动带上 package cn;为了弄清这个问题，我尝试注释它，以观察它的作用。

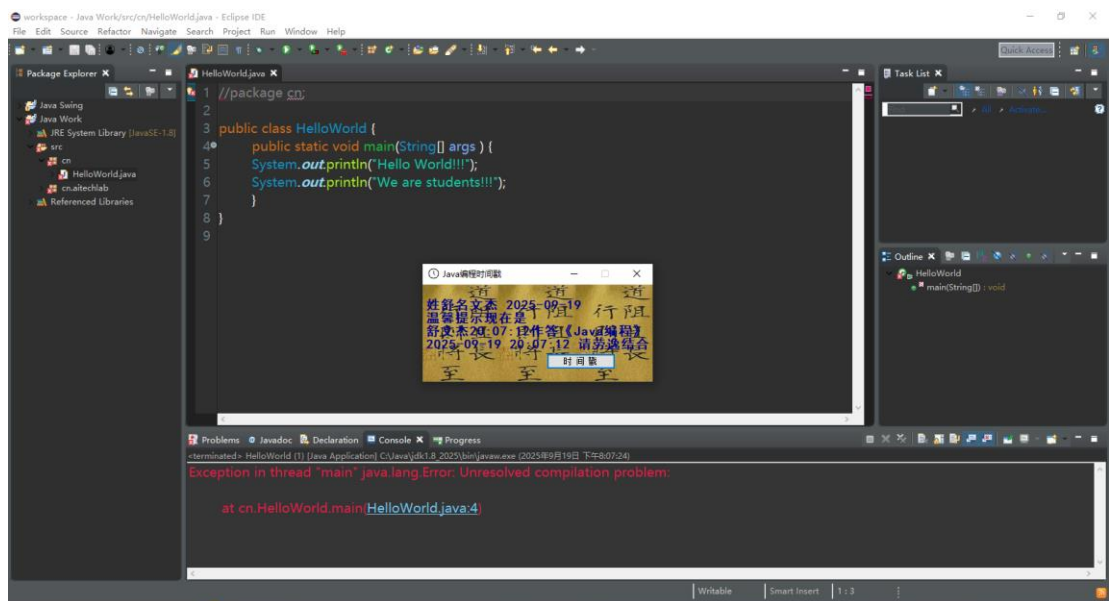


图 5 运行结果报错

？，为什么能在控制台运行的代码无法在 eclipse 软件运行呢？我参照了老师提供的示例代码，发现是少了对包的声明导致的，于是我查阅相关的资料：原来是我创建的类与包的目录不匹配，我必须将文件放到指定的目录下才能正常运行，这里包声明为空，则其会自动匹配到默认包，但该文件是保存在 cn 这个包下的，自然会报错。

因此，为解决这个问题，我重新建立了一个在默认包下的 HelloWorld 类：

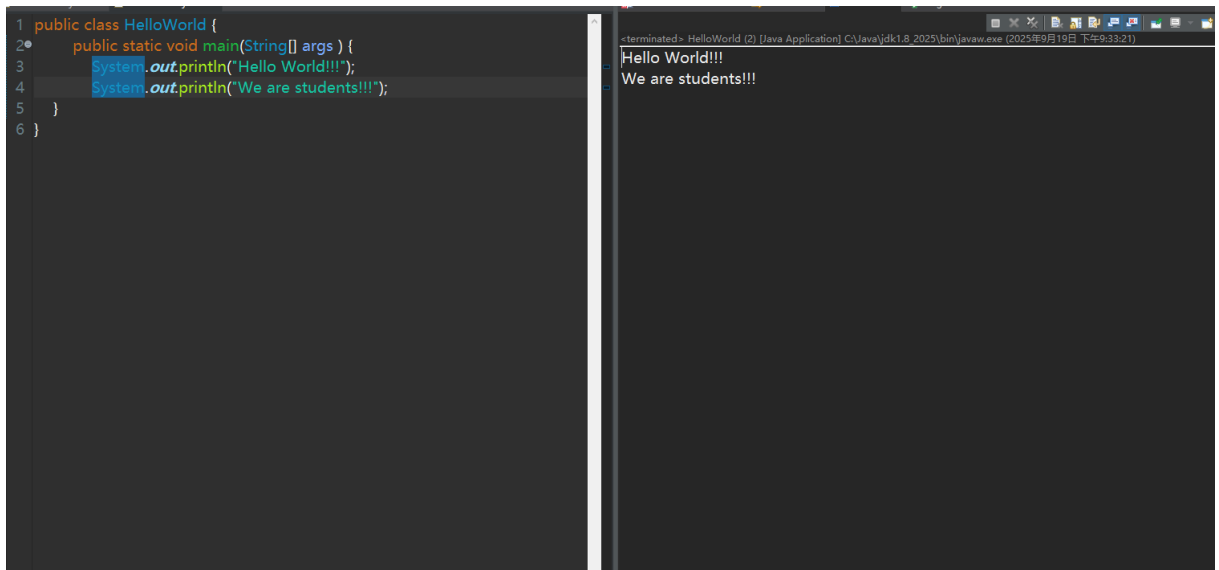


图 6 无包声明的运行成功

终于，我们的代码成功运行，这说明我们的想法是正确的。因此，我怀疑在命令行模式运行 java 代码时，我们的代码实际上是以默认包方式下运行的，但是在大型项目下，包机制保证了良好的模块性与封装性——也就是面向对象而非面向过程的特点。

4) 看到老师的反编译程序，我便想试试它是如何反编译/.class 文件的

一开始，我将其直接放在其所下载的文件夹中，其显示需要对应的 java 环境

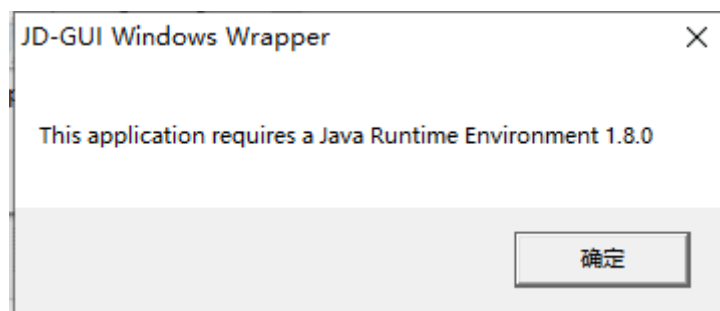


图 7 缺少 Java 环境报错

于是，我将其放置在 C:\Java\jdk1.8_2025 路径下，其终于成功运行。



图 8 反编译结果

为什么反编译工具需要存在 java 环境才能成功运行呢，查询后发现，JD_GUI 本来就是使用 java 语言编写的，要让他们成功运行就需要 JVM 环境，同时由于这个本身就是反编译工具，默认了用户本身就有 JVM 环境，为了尽可能精简，可能没有打包 JVM 进入程序。而我的环境变量在 C:\Java\jdk1.8_2025 路径下，因此得那里进行运行。

1) 编译器怎样提示丢失大括号的错误？

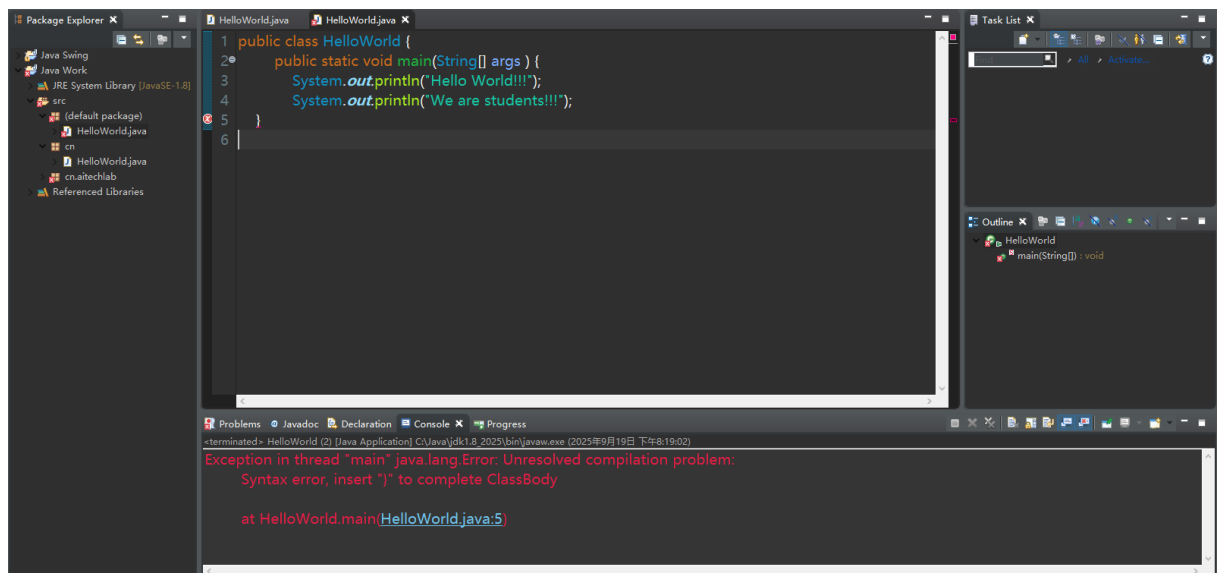


图 9 缺少大括号的报错

报错为 `class, interface, or enum expected`（需在该位置定义类 / 接口 / 枚举，实际因大括号缺失导致结构错误）。

2) 编译器怎样提示语句丢失分号的错误？

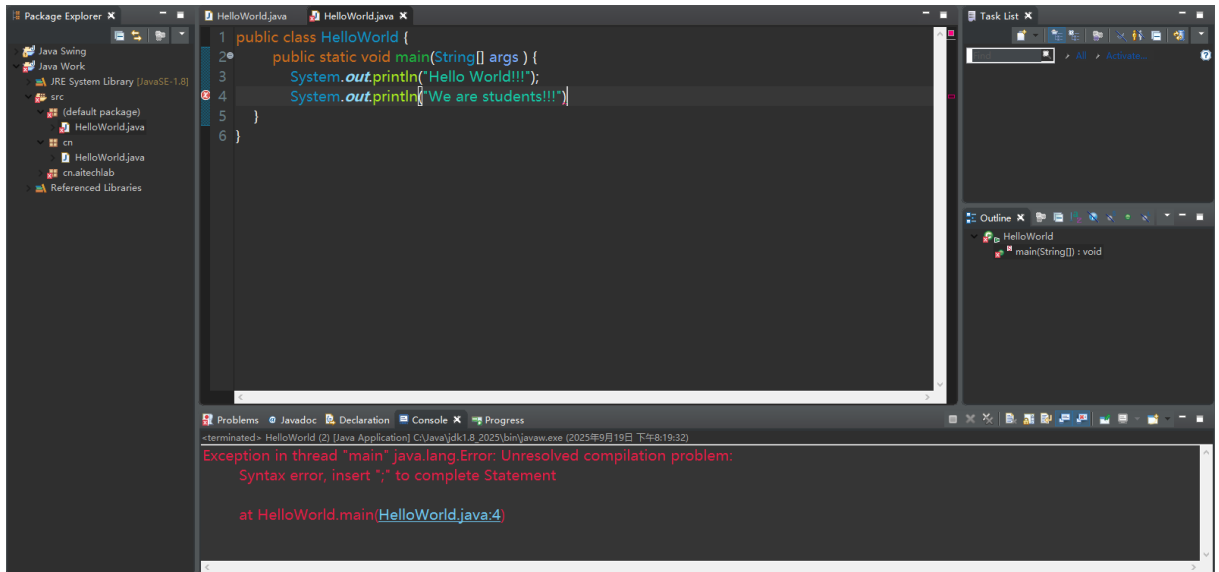


图 10 缺少分号的报错

编译器显示 **Syntax error on token "XXX", ; expected**（在“XXX”标记后需要分号，如变量声明 `int x = 10` 后未加分号，会提示 `; expected`）。这点同 C 语言一样。

3) 编译器怎样提示将 System 写成 system 这一错误？

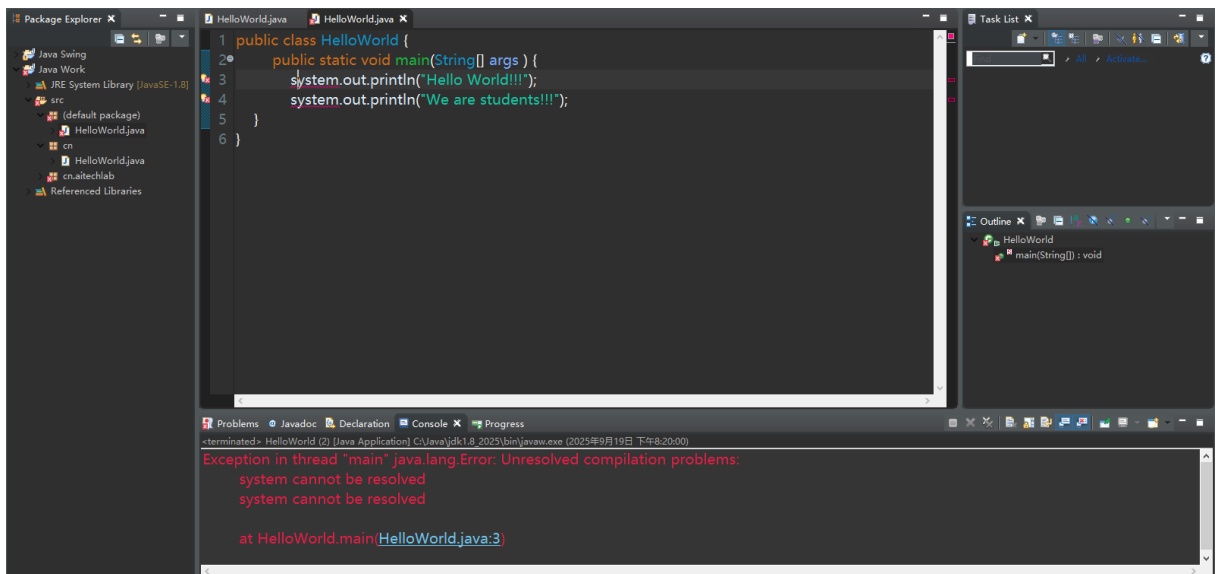


图 11 将大写 System 改为小写 system 的报错

编译器显示 **“Cannot resolve symbol 'system'”**，并定位到错误行。这是因为 Java 是大小写区分的，将大写 S 改为小写将导致其无法匹配对应的类，导致报错。

【实验心得】

一、实验过程体会

1.面向对象的良好封装性

在实验中，由于我们缺少包声明，则代码无法运行，充分体现了 Java 的封装性。当在 Eclipse 中创建类时自动生成 `package cn;` 声明，说明 Java 通过包将类组织成模块化的单元。这与命令行直接运行默认包的形式形成鲜明对比——包能够避免命名冲突，提高代码的可维护性和复用性，这充分展现了面向对象的优点。

二、一次编译，到处运行的进一步理解

在反编译实验中，我对 Java 语言的半编译半解释的特点有了更深的理解，Java 源代码编译后生成 `.class` 字节码文件，需通过 JVM 解释执行。实验中无论是命令行编译运行，还是 Eclipse 内置编译器，最终都依赖 JVM。反编译工具 JD-GUI 无法直接运行而需依赖本地 JDK 环境，进一步印证了 JVM 是 Java 跨平台能力的核心——任何 Java 相关工具本身也需 JVM 支持。

同时我也体会到了环境依赖的重要性，实验中反编译工具仅在 JDK 路径下才能运行，说明 Java 生态中工具链与运行环境紧密关联。这提醒我们需正确配置环境变量（如 `JAVA_HOME`），否则即使看似简单的工具也无法正常工作。

二、概念理解与体会

1、什么是面向对象？什么是面向过程？

面向对象即面向代码中我们要实现的功能，我们主要是要构建一个个类，分解我们面对的实际任务的职责与功能，以类为实现载体；面向过程则是一种线性思维，它常常以步骤与过程来分解任务，构建一个个函数来实现不同的步骤，最后在主程序中运行。

总结来说，面向对象关注对象和职责，将问题分解为相互作用的对象；而面向过程关注步骤和流程，将问题分解为一系列要执行的函数

2、通过下面的五子棋游戏设计来进一步理解面向对象编程的思想和面向过程编程思想的不同之处。试通过其他的例子解释并理解面向对象和面向过程的差异。

五子棋游戏面向过程的设计思路就是首先分析问题的步骤：

1、开始游戏，2、黑子先走，3、绘制画面，4、判断输赢，5、轮到白子，6、绘制画面，7、判断输赢，8、返回步骤 2，9、输出最后结果。把上面每个步骤用分别的函数来

实现，问题就解决了。而面向对象的设计则是从另外的思路来解决问题。整个五子棋可以分为 1、黑白双方，这两方的行为是一模一样的，2、棋盘系统，负责绘制画面，3、规则系统，负责判定诸如犯规、输赢等。

第一类对象（玩家对象）负责接受用户输入，并告知第二类对象（棋盘对象）棋子布局的变化，棋盘对象接收到了棋子的 i 变化就要负责在屏幕上面显示出这种变化，同时利用第三类对象（规则系统）来对棋局进行判定。

面向对象是以功能来划分问题，而不是步骤。面向对象是把构成问题事务分解成各个对象，建立对象的目的不是为了完成一个步骤，而是为了描述某个事物在整个解决问题的步骤中的行为。

例如在我编写的对于一个基带通信系统的 Matlab 仿真程序中，我一开始的构建是面向过程的构建的，因为对于一般的通信系统，我们需要经过以下步骤：

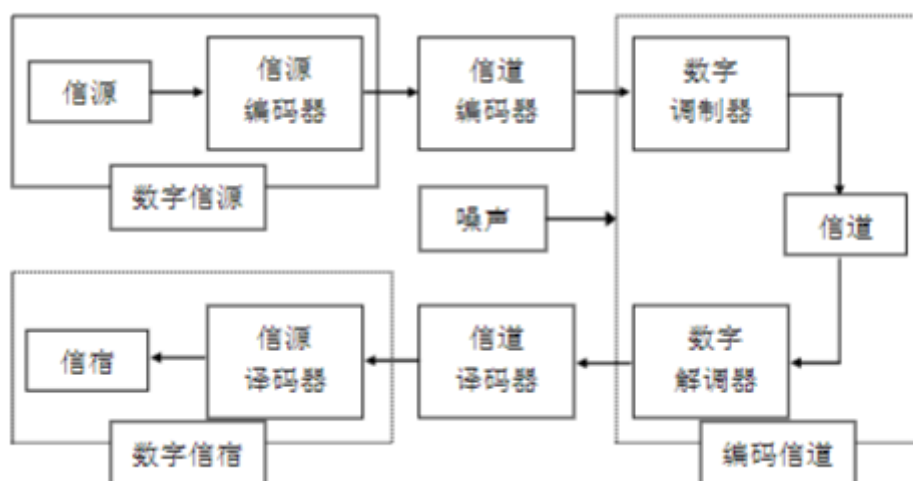


图 12 基带通信系统框图

因此，从面向过程的角度来说，我想要仿真一个这样的系统，我们需要先进行信源编码，波形成型，数字调制，加入信道噪声，载波同步与解调，定时判决等步骤，这是一个线性的流程，因此上述步骤我们常用函数来实现。

但是学习了面向对象的思想后，如果我们想使用面向对象的角度来仿真该通信系统，我们则会将这个通信系统分为三个主要的角色——信源，信道与信宿，构建这三大类，并通过这三大类的数据交互来实现对于该通信系统的仿真。（另一个例子是在实现电路中，我们常常也是面向对象，令不同的电路承担不同的职责，电气参数如数据一样在不同对象交互）

3、简单说说面向对象的基本概念和面向对象程序设计的特点和优点

1. 面向对象编程即面向任务中不同对象的关系与职责，我们主要关注在不同对象的数据交互；其特点是以对象为中心，以数据为主导，对象间通过消息传递进行通信，一个对象通过调用另一个对象的方法来请求其执行某个操作。具有良好的封装性。优点在于在构建一个复杂系统时，面向对象可以最大程度复用过往的代码，其可拓展性较强，同时，其封装性较好，非常适合多人进行联合开发。